



TIC-TAC-TOE

Von Fabio Stoffels,
Leal de Suza, Peter
Kretzer

PROJEKTÜBERBLICK

Klassisches Tic-Tac-Toe für zwei Spieler

Desktop-Anwendung mit Java Swing

Klare Trennung von Logik und Oberfläche

Keine externen Bibliotheken

ZIEL DES PROJEKTS



Umsetzung eines bekannten Spiels in Java



Fokus auf saubere Objektorientierung



Trennung von Spiellogik, GUI und Steuerung



Gut erweiterbarer und übersichtlicher Code

TECHNISCHE RAHMENBEDINGUNGEN



Java SE 17 oder höher



Reines Standard-Java (kein Maven/Gradle)

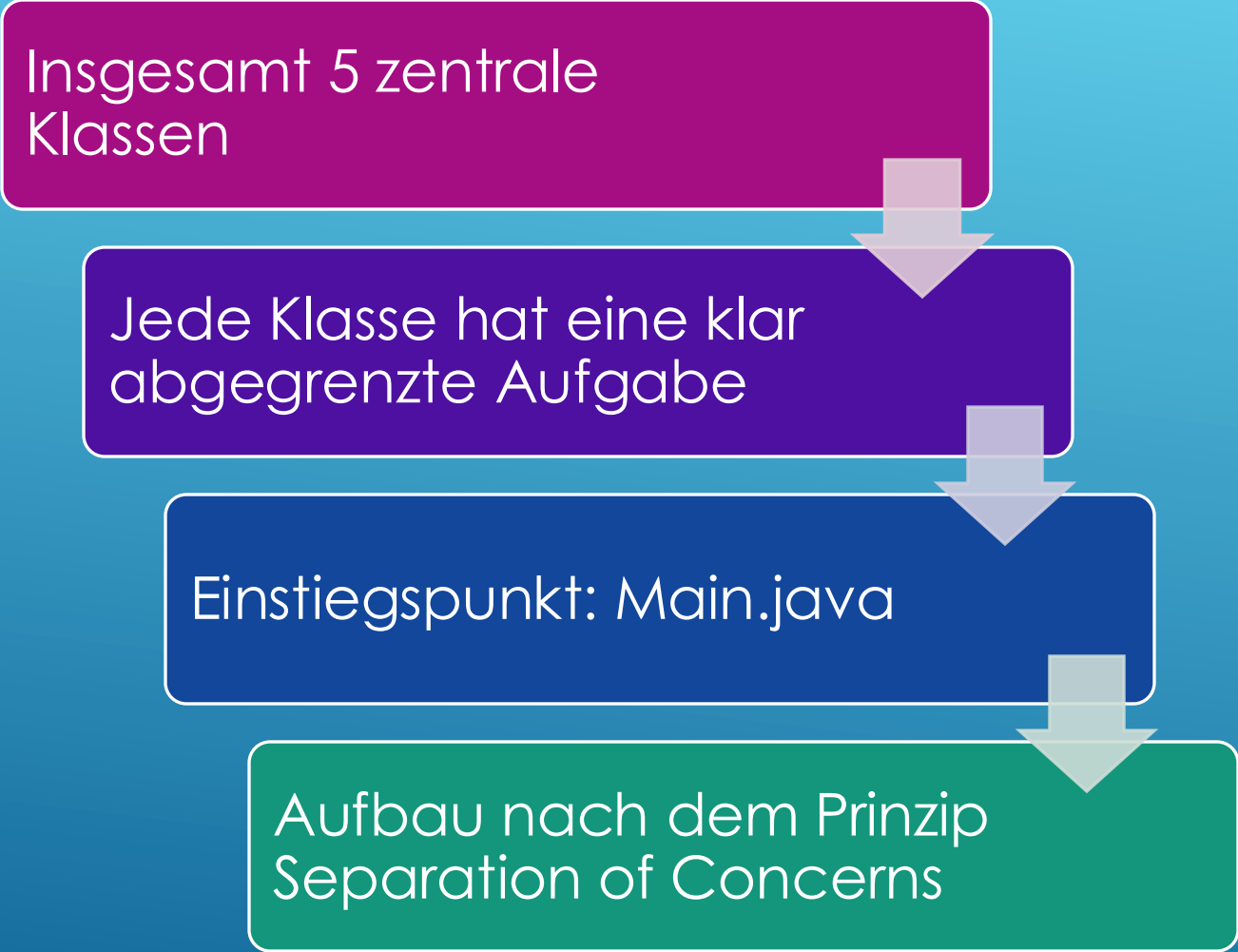


Start über Konsole möglich



Plattformunabhängig (Windows, macOS, Linux)

Insgesamt 5 zentrale
Klassen



```
graph TD; A[Insgesamt 5 zentrale Klassen] --> B[Jede Klasse hat eine klar abgegrenzte Aufgabe]; B --> C[Einstiegspunkt: Main.java]; C --> D[Aufbau nach dem Prinzip Separation of Concerns];
```

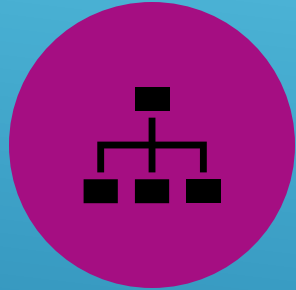
Jede Klasse hat eine klar
abgegrenzte Aufgabe

Einstiegspunkt: *Main.java*

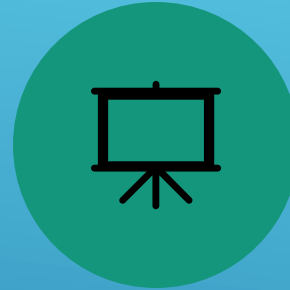
Aufbau nach dem Prinzip
Separation of Concerns

PROJEKTSTRUKTUR





Model: Spiellogik und Zustand



View: Grafische Darstellung (Spielfeld, Fenster)

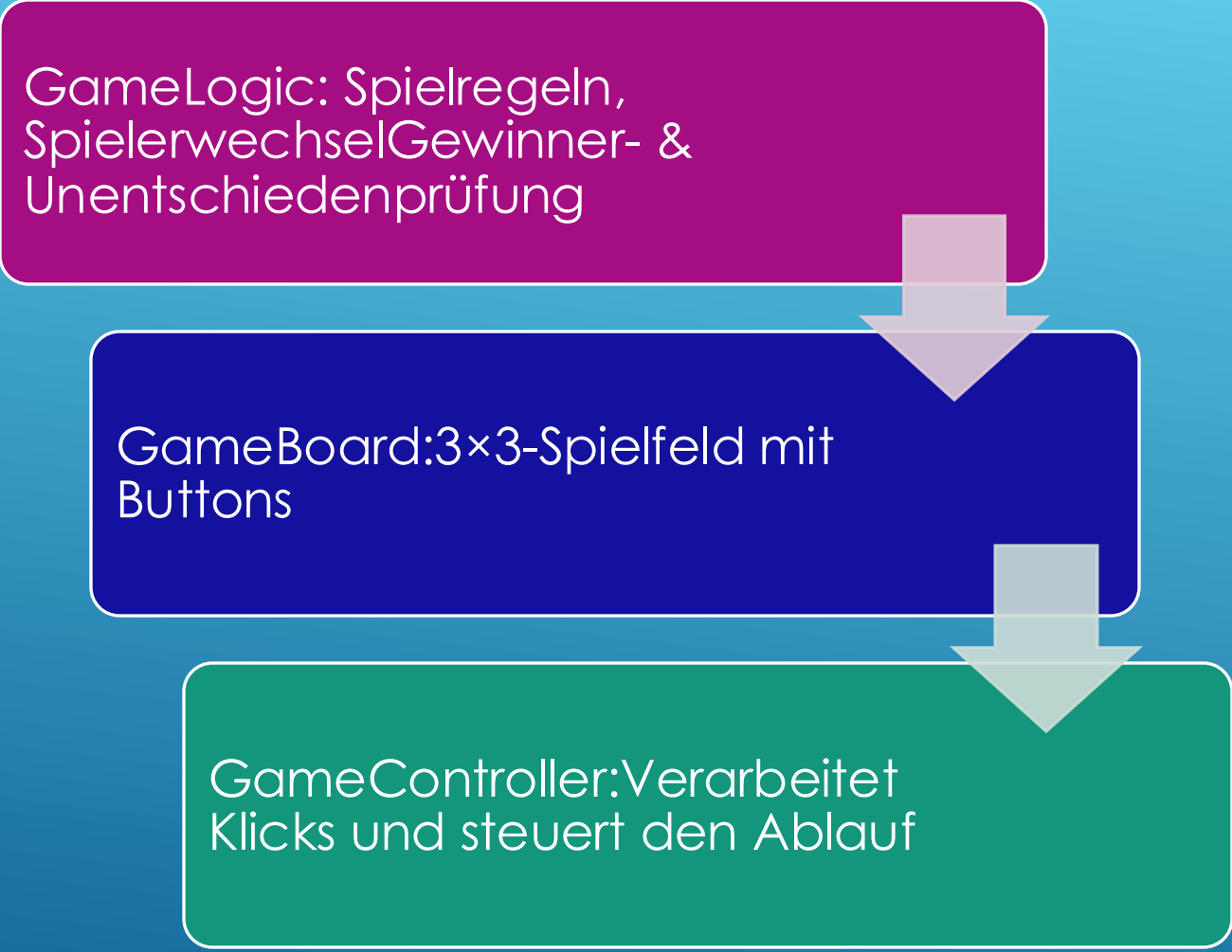


Controller: Verbindet Klicks mit der Logik



Vorteil: Bessere Wartbarkeit & Testbarkeit

GameLogic: Spielregeln,
SpielerwechselGewinner- &
Unentschiedenprüfung



```
graph TD; A[GameLogic: Spielregeln, SpielerwechselGewinner- & Unentschiedenprüfung] --> B[GameBoard: 3x3-Spielfeld mit Buttons]; B --> C[GameController: Verarbeitet Klicks und steuert den Ablauf];
```

GameBoard: 3×3-Spielfeld mit
Buttons

GameController: Verarbeitet
Klicks und steuert den Ablauf

KLASSEN

```
/**
 * Setzt alle Buttons auf ihren Ausgangszustand zurück (leer, aktiv).
 */
public void felderZuruecksetzen() { 1 usage  👤 Peter Kretzer
    for (int zeile = 0; zeile < GameLogic.GITTER_GROESSE; zeile++) {
        for (int spalte = 0; spalte < GameLogic.GITTER_GROESSE; spalte++) {
            aktualisiereButton(zeile, spalte, PlayerSymbol.LEER);
        }
    }
}
```



```
Debeka@Peter MINGW64 ~/IdeaProjects/Java_Repo (main)
```

```
$ git merge Symbols-Colors main
```

```
Merge made by the 'ort' strategy.
```

TicTacToe/src/GameBoard.java		14	++-----
TicTacToe/src/GameController.java		8	-----
TicTacToe/src/GameWindow.java		16	+++-----
TicTacToe/src/PlayerSymbol.java		6	-----
out/production/TicTacToe/GameBoard.class		Bin	3753 -> 3750 bytes
out/production/TicTacToe/GameWindow.class		Bin	6731 -> 6721 bytes

6 files changed, 5 insertions(+), 39 deletions(-)

Spieler klickt auf
ein Feld



Controller
wertet Klick aus



Spielfeld und
Statusanzeige
werden
aktualisiert



Logik prüft
Gültigkeit &
Spielende



ERWEITERUNGEN

KI-Gegner (z. B.
Minimax-Algorithmus)

Spielstatistiken (Siege /
Niederlagen)

Größeres Spielfeld (z. B. 4×4)

Sound- oder Animationseffekte

Fazit



Überschaubares, aber
vollständiges Projekt



Gute Umsetzung der
MVC-Architektur



Sauberer, verständlicher Code



Ideale Basis für spätere
Erweiterungen